Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Laboratory Work 3

The Command Line: Shell, Pipes and Text Filters

Operating Systems

Done by: st. gr. FAF-241, Cațț Chiril

Checked by:
asist. univ. Reițman Patricia

Chișinău – 2026

Contents

Introduction **1**

Objective 1

Environment and evidence 1

1 Move Fast and Find Help **1**

2 Shell Expansion and Quoting **3**

3 Redirection, Chaining and Jobs **4**

4 Questions Answered with Text Filters **5**

5 The Report Script **5**

6 Discussion **6**

7 Conclusion **6**

Introduction

The shell interprets a command line before starting a program. It completes paths, expands patterns and variables, applies quoting and redirections, starts processes, and connects them through pipes. This laboratory examines those steps in Bash and uses small text filters to answer questions about a generated access log.

Objective

The objective is to navigate efficiently in Bash, predict shell expansion, control standard input and output, combine commands, and write executable scripts that summarize structured text.

Environment and evidence

The supplied terminal captures show a Linux virtual machine with user `class`. They document the data setup and Part 1 observations. The generated access log follows the exact loop in the lab instructions, so its numerical results can be reproduced. Later command behavior is explained from Bash syntax and the scripts accompanying this report; machine-dependent outputs are left as such rather than attributed to an unseen VM session.

1 Move Fast and Find Help

Tab completed `ls /usr/sh` to the unique directory prefix `/usr/share/`, as shown in Figure 1.1. When several names match, two Tab presses list possible completions. Bash completion saves typing and reduces mistakes in long paths.

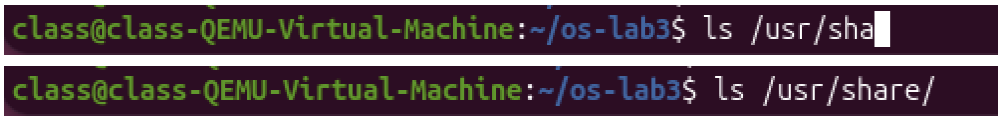


Figure 1.1 – Tab completion of a directory path

The reverse search shortcut `Ctrl+R` searched command history for `whoami`. Enter would execute the matching history entry; editing the line first is also possible.

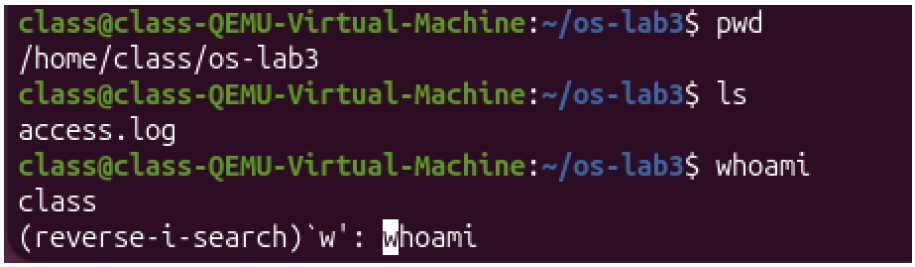


Figure 1.2 – Reverse search finding an earlier command

The `type` command identifies `cd` and `echo` as Bash built-ins. In this VM, `ls` is an alias for `ls -color=auto`; `which ls` finds its external executable at `/usr/bin/ls`. The shell must execute `cd` itself because changing the working directory of a child process would not change the parent shell's directory.

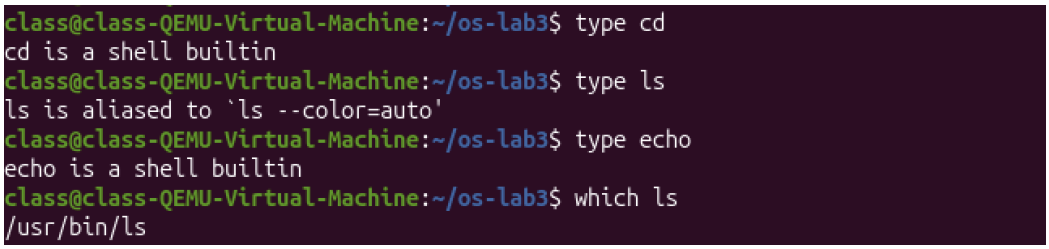


Figure 1.3 – The type and which commands identify built-ins, an alias, and the ls executable

The captured `PATH` contains ten entries, including a repeated `/snap/bin`; the number of distinct directories is therefore nine. Bash searches these entries in order, and `/usr/bin` contains `ls`. A missing command returned status 127. For help, `man ls` and `ls -help` show that `-t` sorts by modification time; `help cd` explains the built-in command. The command `man -k "list directory"` searches manual-page descriptions.



Figure 1.4 – The ten PATH entries in the Linux virtual machine

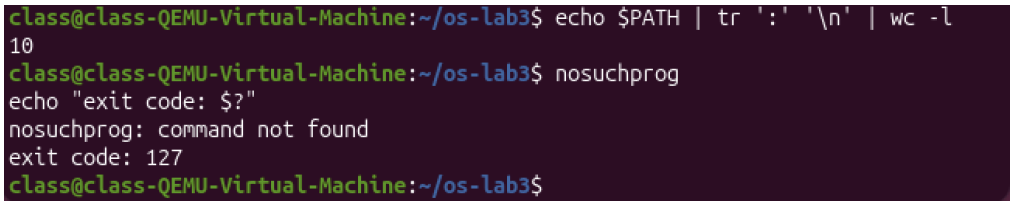


Figure 1.5 – A missing program returning exit code 127

2 Shell Expansion and Quoting

The test directory contains `a.c`, `b.c`, `file1.txt`, `file2.txt`, `notes.txt`, and `alfa beta.txt`. Table 1 records the predicted behavior and reproducible Bash result. For environment-dependent expansions, the table gives the output form rather than an invented path or user name.

Table 1: Predictions and Bash results for shell expansion

Command	**Prediction**	**Result and explanation**
`echo *.c`	C source names	`a.c b.c`; Bash expands the glob before `echo`.
`echo file?.txt`	Both numbered files	`file1.txt file2.txt`; `?` matches one character.
`echo [a-b]*`	Names beginning a or b	`a.c alfa beta.txt b.c`; the name with a space remains one argument.
`echo *.xyz`	Literal pattern	`*.xyz`; unmatched globs remain unchanged in default Bash.
`ls alfa beta.txt`	Two lookup errors	Bash passes two names; neither `alfa` nor `beta.txt` exists.
`ls "alfa beta.txt"`	The one matching file	Bash passes the complete name as one argument.
`echo '$HOME' "$HOME" \$HOME`	Literal, value, literal	First and third tokens are `$HOME`; the middle token is the home path.
`echo {1..5}; echo $((7 * 6))`	A sequence and arithmetic	`1 2 3 4 5`, then `42`.

Command	Prediction	Result and explanation
`echo "today is $(date +%A), I am $(whoami)"`	Day and user	The weekday and login name are substituted; they depend on execution time and user.
`touch log{1..3}.txt && ls log*`	Three log names	`log1.txt log2.txt log3.txt`; the listing runs if `touch` succeeds.

The `echo` program does not search the directory: Bash performs pathname expansion before calling it. `ls alfa beta.txt` can be fixed with either `ls "alfa beta.txt"` or `ls alfa\ beta.txt`. Single quotes preserve characters literally; double quotes allow variable and command substitution but preserve spaces within one argument. If `ls` prints `'alfa beta.txt'`, those visible quotes are its display convention for an unusual filename; they are not part of the filename.

3 Redirection, Chaining and Jobs

Standard output is file descriptor 1 and standard error is file descriptor 2. With `ls /etc /nope > out.txt 2> err.txt`, the valid directory listing goes to `out.txt` while the missing-path error goes to `err.txt`. Thus `wc -l out.txt` counts listing lines and `cat err.txt` shows the error.

The order of redirections matters. In `> all.txt 2>&1`, Bash first points stdout at the file and then duplicates that destination for stderr, so both streams enter the file. In `2>&1 > all.txt`, stderr is first duplicated from the original stdout, usually the terminal; stdout is then changed to the file. The error stays on the terminal while the listing enters `all.txt`.

The two writes `echo first > notes.md` and `echo second » notes.md` leave the file with two lines, because `>` truncates and `»` appends. The command `wc -l < access.log` prints only `500`, while `wc -l access.log` prints `500 access.log`: in the first case `wc` receives bytes on stdin and no filename argument.

The separator `;` runs the next command regardless of status. The operator `&&` runs it only if the preceding command succeeded, and `||` runs it only if the preceding command failed. For example, `ls /nope 2> /dev/null || echo "failed with code $?"` prints the failure status from `ls`. Finally, `sleep 100 &` starts a background job; `jobs` lists it and `kill %1` sends a termination signal to job 1. The precise job-status text depends on timing.

4 Questions Answered with Text Filters

The provided generator writes exactly 500 rows with fields *date*, *time*, *user*, *result*, and *page*. The terminal capture confirms the row count.

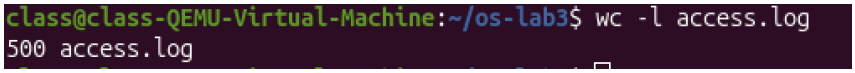


Figure 4.1 – The generated access log contains 500 requests

The companion `oneliners.sh` contains one command line for each question. Table 2 summarizes the deterministic answers; question 7 depends on the machine's `/etc/passwd` and is printed by the script when it runs in the VM.

Table 2: Answers derived from the generated access log

Q	Answer
1	100 failed requests.
2	Four pages: `/page0`, `/page1`, `/page2`, `/page3`.
3	Each page received 125 requests; the counts sum to 500.
4	`user3` and `user5` tie for the most failures, with 15 each.
5	`user3` made 72 requests, 15 of which failed.
6	The last three failed requests are `10:10 user0`, `10:15 user5`, and `10:20 user3`.
7	The login-shell counts are environment-specific; the script extracts field 7 of active `/etc/passwd` records and counts equal paths.
8	On the intended GNU/Linux system, `ls -l /etc` has one extra `total` header line; `ls /etc` does not.

For Q8, the predicted difference is one line: the long listing's `total` header. Piping `ls` makes ordinary output one entry per line, so `wc -l` can count it. This explanation assumes `/etc` is a directory as in the course Linux VM. The optional regular-expression query for failed requests at minutes ending in zero yields 50 rows, because every tenth generated request fails and its minute is a multiple of ten.

5 The Report Script

The accompanying `report.sh` accepts a log path, rejects a missing file, summarizes all requests and failures, and loops over `user0` through `user6`. With the generated dataset, its output is:

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

The user counts add up to 100. Running `./report.sh nope.log` writes `no such file: nope.log` to stderr and exits with status 1. The first line, `#!/bin/bash`, tells the operating system to invoke Bash when the file is executed directly. Before `chmod +x report.sh`, `./report.sh` fails with a permission error, though `bash report.sh` can still read it. The `./` prefix specifies the file in the current directory, which is normally absent from `PATH`. Writing errors with `>&2` separates them from ordinary output, and `exit 1` makes failure observable to callers.

6 Discussion

The shell, rather than individual programs, performs completion, history lookup, globbing, quoting, substitutions, and redirection setup. Programs such as `echo`, `ls`, `grep`, and `wc` then receive already-prepared arguments or streams. The access-log exercise illustrates why pipelines work well for text data: each command performs one small transformation, and the counts can be checked against the generator's 500 rows.

The available screenshots document setup and Part 1. The Part 2–5 explanations and script results are reproducible, but terminal screenshots for those sections were not supplied. The contents of `/etc/passwd`, the exact `/etc` listing count, and job-control messages should be captured from the course VM if those figures are required for submission.

7 Conclusion

This work demonstrated how Bash interprets a command line, locates commands, expands patterns, controls file descriptors, and connects simple filters. The generated log contained 500 requests and 100 failures; all page counts and per-user failure counts reconcile with those totals. The two executable scripts preserve the one-line answers and extend them into a reusable report with explicit error handling.